

Comprehensive Analysis and Practical Application of Git Version Control

Ved Jignesh Parmar

Driverless Formula Manipal

Computer Science and Engineering

Manipal Institute of Technology

Manipal, India

Abstract—This report provides an in-depth exploration of Git, a distributed version control system. It covers core architectural concepts such as commits and object storage, branching strategies, and repository synchronization methods including pushing and pulling. Advanced operations such as rebasing, resolving merge conflicts, reference logs, and stashing are also examined. Furthermore, the report outlines a practical demonstration involving local repository management and remote synchronization, fulfilling the requirements for comprehensive version control proficiency.

I. INTRODUCTION

Version control systems are foundational to modern software engineering, enabling multiple developers to collaborate on a codebase simultaneously without destructive interference. Git has emerged as the industry standard due to its distributed nature, speed, and robust branching model. Before collaborating, developers initialize their environment using the git config command to set user credentials, and obtain remote repositories locally via git clone. This report details the internal mechanics of Git and demonstrates practical workflows, including the management of local environments and integration with remote repositories.

II. CORE GIT ARCHITECTURE

A. Commits and Storage

Unlike delta-based version control systems, Git conceptualizes its data as a stream of snapshots. Prior to committing, developers use git status to check what has been added and git diff to view unstaged modifications. Every time a commit is made (Format: git commit -m "<commit message>"), Git takes a picture of what all files look like at that moment and stores a reference to that snapshot. If files have not changed,

Git simply links to the previous identical file it has already stored. Internally, Git stores these as objects to efficiently manage and track repository data over time.

B. Logs and Reference Logs

The git log command (Format: git log) allows developers to traverse the commit history of a branch. When navigating history, the HEAD pointer serves as a reference to the current commit. Developers can move relative to HEAD using caret and tilde symbols, such as HEAD^ for the immediate parent or HEAD~3 to reference three commits ago. Git also maintains a reflog (Format: git reflog), which records updates to the tip of branches and other references in the local repository. The reflog is crucial for recovering "lost" commits that are no longer reachable from any branch.

III. BRANCHING AND SYNCHRONIZATION

A. Branching

Branching in Git is highly lightweight, simply involving a movable pointer to a specific commit. The command to create a branch is git branch <branch-name>, and to switch to it, git checkout <branch-name>. This encourages frequent branching for features, bug fixes, and experiments, allowing developers to isolate changes from the primary production codebase.

Forced Branch Assignment: To forcefully move a branch pointer directly to a specific commit, the force flag is used (Format: git branch -f <branch-name> <commit-hash>).

B. Remote Operations: Push and Pull

Synchronization with remote repositories is handled via pulling (Format: `git pull <remote> <branch>`), which fetches data and merges it into the local branch, and pushing (Format: `git push <remote> <branch>`), which sends local commits to the remote. While pull automatically integrates changes, developers can use `git fetch` to safely download remote updates without modifying their local branches. Force pushing (Format: `git push -f`) overwrites the remote branch history with the local history.

IV. INTEGRATION AND CONFLICT MANAGEMENT

A. Merging vs. Rebasing

Integration of branches can be achieved via merging or rebasing. Merging (Format: `git merge <branch-name>`) creates a new "merge commit" that ties the histories of two branches together. Rebasing (Format: `git rebase <branch-name>`) takes the commits from one branch and replays them sequentially on top of another. Alternatively, `git cherry-pick <commit>` allows developers to copy one specific commit from another branch onto their current branch.

Interactive Rebasing: For fine-grained control over commit history, the interactive flag is utilized (Format: `git rebase -i <commit-hash>`). This command opens an editor allowing developers to squash, edit, reorder, or drop specific commits.

B. Resolving Merge Conflicts

Conflicts arise when the same part of a file is modified differently in two branches being integrated. Git halts the process and marks the conflicted files. Developers must manually edit these files to choose the correct code, stage the resolved files (Format: `git add <file-name>`), and complete the commit process.

C. Discarding Changes and Stashing

To discard unwanted modifications, `git restore` deletes unstaged changes to a file, while `git clean` permanently removes untracked files from the working directory. If changes are meant to be kept but not yet committed, the `git stash` command temporarily shelves changes made to a working copy so the developer can switch branches. Stashed changes can be reapplied later using `git stash pop`.

V. PRACTICAL DEMONSTRATION

As part of the practical application, the interactive tutorials on learngitbranching.js.org were thoroughly completed.

Following the simulation, a local Git repository was initialized, and I committed Task-1 codes to the repository.

VI. COMPREHENSIVE GIT COMMAND REFERENCE

The following is a complete categorization of essential Git commands adapted from the official Git Cheat Sheet [1]. All commands use italics rather than bold text to align with strict IEEE formatting standards.

A. Getting Started

git init

Start a new repo [1].

git clone <url>

Clone an existing repo [1].

B. Prepare to Commit

git add <file>

Add untracked file or unstaged changes [1].

git add .

Add all untracked files and unstaged changes [1].

git add -p

Choose which parts of a file to stage [1].

git mv <old> <new>

Move file [1].

git rm <file>

Delete file [1].

git rm --cached <file>

Tell Git to forget about a file without deleting it [1].

git reset <file>

Unstage one file [1].

git reset

Unstage everything [1].

git status

Check what you added [1].

C. Make Commits

git commit

Make a commit (and open text editor) [1].

git commit -m 'message'

Make a commit with message inline [1].

git commit -am 'message'

Commit all unstaged changes [1].

D. Move Between Branches

git switch <name> OR git checkout <name>
Switch branches [1].

git switch -c <name> OR git checkout -b <name>
Create a branch [1].

git branch
List branches [1].

git branch --sort=-committerdate
List branches by most recently committed to [1].

git branch -d <name>
Delete a branch [1].

git branch -D <name>
Force delete a branch [1].

E. Diff Staged/Unstaged Changes

git diff HEAD
Diff all staged and unstaged changes [1].

git diff --staged
Diff just staged changes [1].

git diff
Diff just unstaged changes [1].

F. Diff Commits

git show <commit>
Show diff between a commit and its parent [1].

git diff <commit> <commit>
Diff two commits [1].

G. Ways to Refer to a Commit

References
You can use a branch name, a tag, a commit ID, 'main', 'origin/main', 'HEAD', or offsets like 'HEAD~3' [1].

H. Discard Your Changes

git restore <file> OR git checkout <file>
Delete unstaged changes to one file [1].

git restore --staged --worktree <file> OR git checkout HEAD <file>
Delete all staged and unstaged changes to one file [1].

git reset --hard
Delete all staged and unstaged changes [1].

git clean
Delete untracked files [1].

git stash
Stash all staged and unstaged changes [1].

I. Edit History

git reset HEAD^
Undo the most recent commit (keep working directory same) [1].

git rebase -i HEAD~5
Squash the last 5 commits into one (change 'pick' to 'fixup') [1].

git reflog BRANCHNAME THEN git reset --hard <commit>
Undo a failed rebase manually finding the commit ID [1].

git commit --amend
Change a commit message (or add a file you forgot) [1].

J. Code Archaeology

git log main
Look at a branch's history [1].

git log --graph main
Look at a branch's history with graph visualization [1].

git log --oneline
Look at a branch's history in one line format [1].

git log <file>
Show every commit that modified a file [1].

git log --follow <file>
Show every commit that modified a file, including before it was renamed [1].

git log -G banana
Find every commit that added or removed some text ('banana') [1].

git blame <file>
Show who last changed each line of a file [1].

git diff <commit> <file>
Diff one file since a commit [1].

git diff <commit> --stat
Show a summary of a diff [1].

git show <commit> --stat
Show a summary of a commit's diff [1].

K. Combine Diverged Branches

git rebase main

Combine diverged branches with rebase [1].

git merge banana

Bring a branch up to date with another branch (fast-forward merge) or combine with standard merge [1].

git merge --squash banana

Combine with squash merge [1].

git cherry-pick <commit>

Copy one commit onto the current branch [1].

L. Push Your Changes

git push origin main

Push the main branch to the remote origin [1].

git push

Push the current branch to its remote tracking branch [1].

git push -u origin <name>

Push a branch that you've never pushed before [1].

git push --force-with-lease

Force push securely [1].

git push --tags

Push tags [1].

M. Pull Changes

git fetch origin main

Fetch changes (but don't change any of your local branches) [1].

git pull --rebase

Fetch changes and then rebase your current branch [1].

git pull origin main OR git pull

Fetch changes and then merge them into your current branch [1].

N. Important Files

.git/config

Local git config [1].

~/.gitconfig

Global git config [1].

.gitignore

List of files to ignore [1].

O. Restore an Old File

git checkout <commit> <file> OR git restore <file> --source <commit>

Get the version of a file from another commit [1].

P. Add a Remote

git remote add <name> <url>

Add a remote repository [1].

Q. Configure Git

git config user.name 'Your Name'

Set a config option [1].

git config --global

Set option globally [1].

git config alias.st status

Add an alias [1].

man git-config

See all possible config options [1].

VII. CONCLUSION

Mastering Git requires understanding both its high-level commands and its underlying object model. Through theoretical study and practical exercises, proficiency in branching, conflict resolution, and remote synchronization has been achieved, laying the groundwork for robust collaborative software development.

REFERENCES

[1] Git, "Git Cheat Sheet," Official Git Site. [Online]. Available: <https://git-scm.com/cheat-sheet>.